

CS4225/CS5425 Big Data Systems for Data Science

NoSQL and Distributed Databases

Bingsheng He
School of Computing
National University of Singapore
hebs@comp.nus.edu.sg



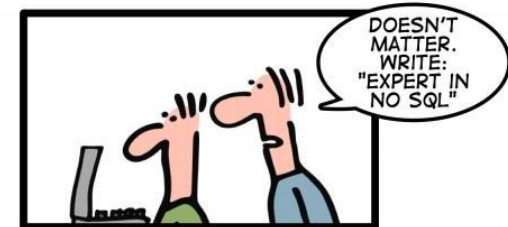
School of Computing¹

Learning Outcomes

1. What is NoSQL?

2. Major types of NoSQL systems

HOW TO WRITE A CV



Leverage the NoSQL boom

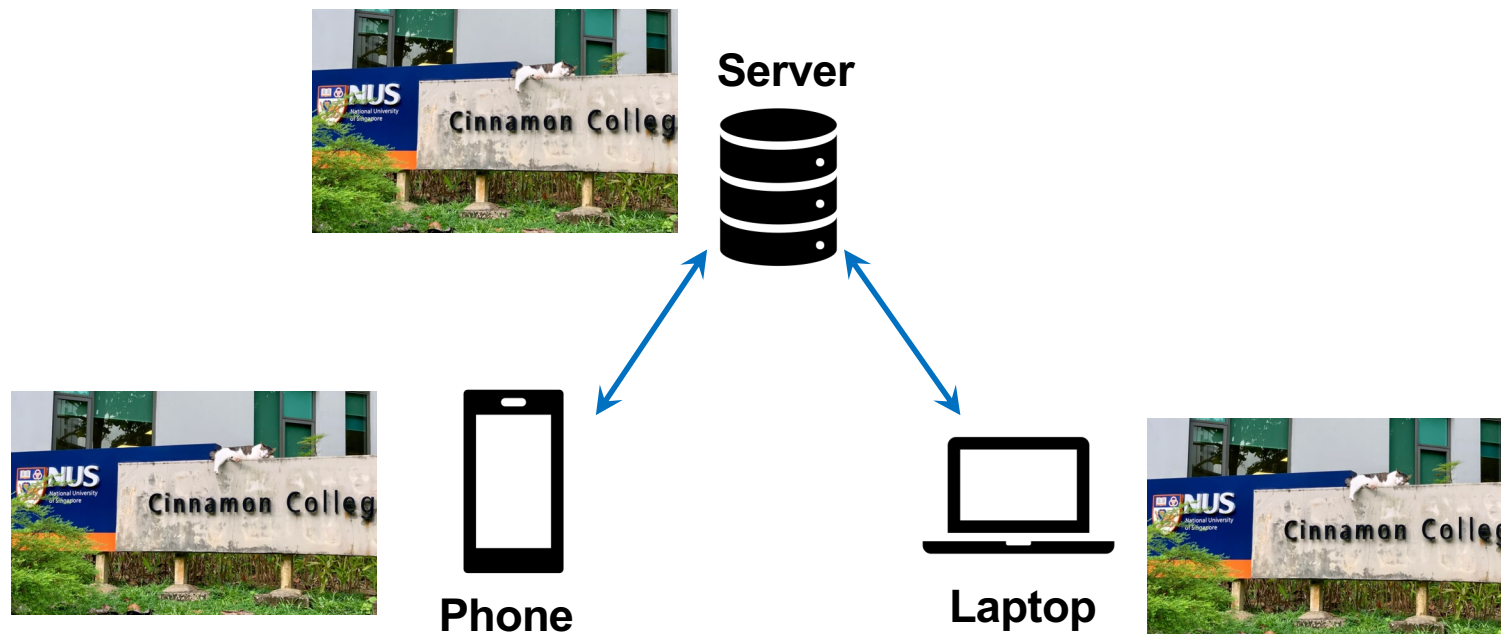
memecenter.com

Example: Photos Synchronization System

- **Goal:** design a system like iCloud / Google Photos that synchronizes photos across each user's devices (phone, computer, etc.)

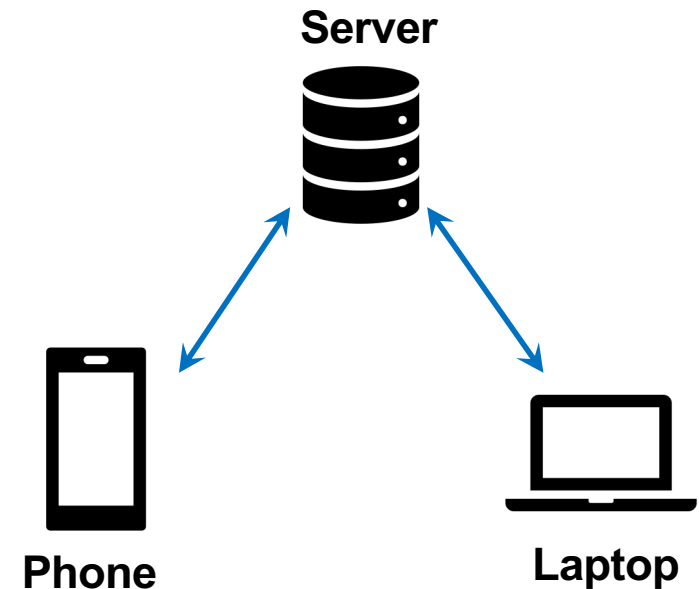
Example: Photos Synchronization System

- **Goal:** design a system like iCloud / Google Photos that synchronizes photos across each user's devices (phone, computer, etc.)



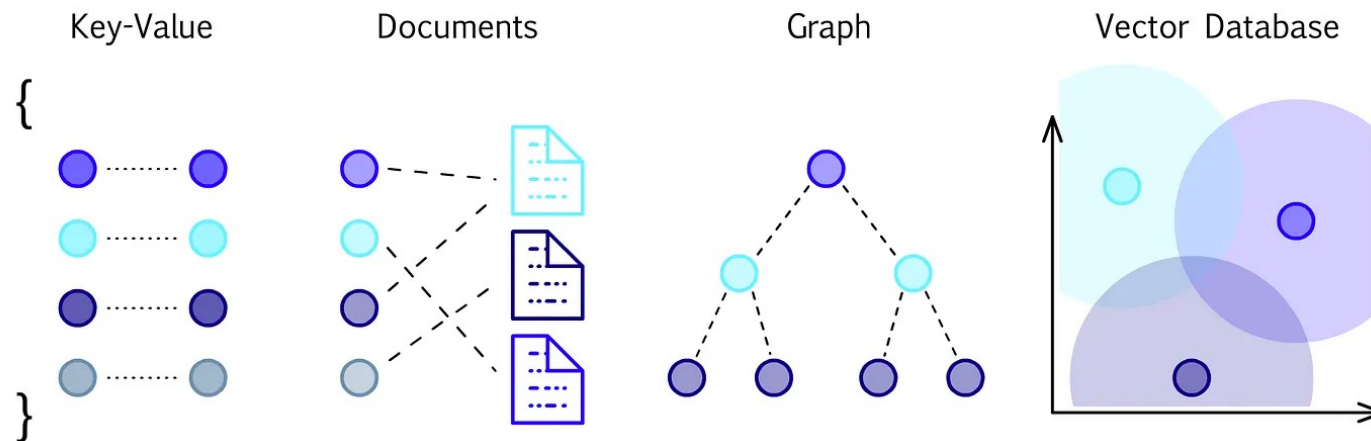
Example: Photos Synchronization System

- **Goal:** design a system like iCloud / Google Photos that synchronizes photos across each user's devices (phone, computer, etc.)
- **Many challenges in practice:**
 - **Connectivity:** devices may lose connection to the internet
 - Data can go “out of sync” between devices
 - May lead to conflicting files / conflicting edits
 - **Eventual consistency:** files should become consistent assuming devices are connected to the internet for a sufficiently long time

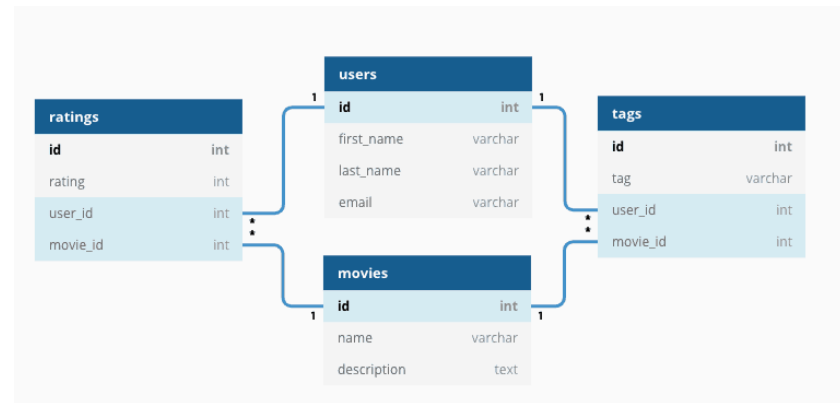


What is NoSQL?

- NoSQL mainly refers to a non-relational database, i.e. it stores data in a format other than relational tables
- "SQL" here refers to traditional relational database management systems ("DBMS")
 - (This is a slight misnomer; here "SQL" is used to refer to relational databases, not the querying language. In fact, NoSQL systems can involve SQL-like querying languages in many cases.)
- NoSQL has come to stand for "Not Only SQL", i.e. using relational and non-relational databases alongside one another, each for the tasks they are most suited for



What is NoSQL?



Traditional Relational Database Management System (DBMS)

```
>>> db.inventory.find()
[
  {
    _id: ObjectId("6138d8b1611d9dc433ad73dd"),
    item: 'canvas',
    qty: 100,
    tags: [ 'cotton' ],
    size: { h: 28, w: 35.5, uom: 'cm' }
  },
  {
    _id: ObjectId("6138d917611d9dc433ad73de"),
    item: 'table',
    qty: 5,
    price: 10
  }
]
```

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

Metaphor: NoSQL = No Rules?



Relational Databases

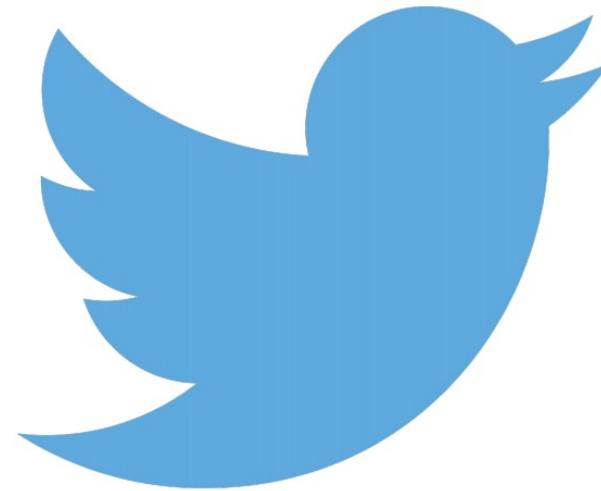


Non-Relational Databases (NoSQL)

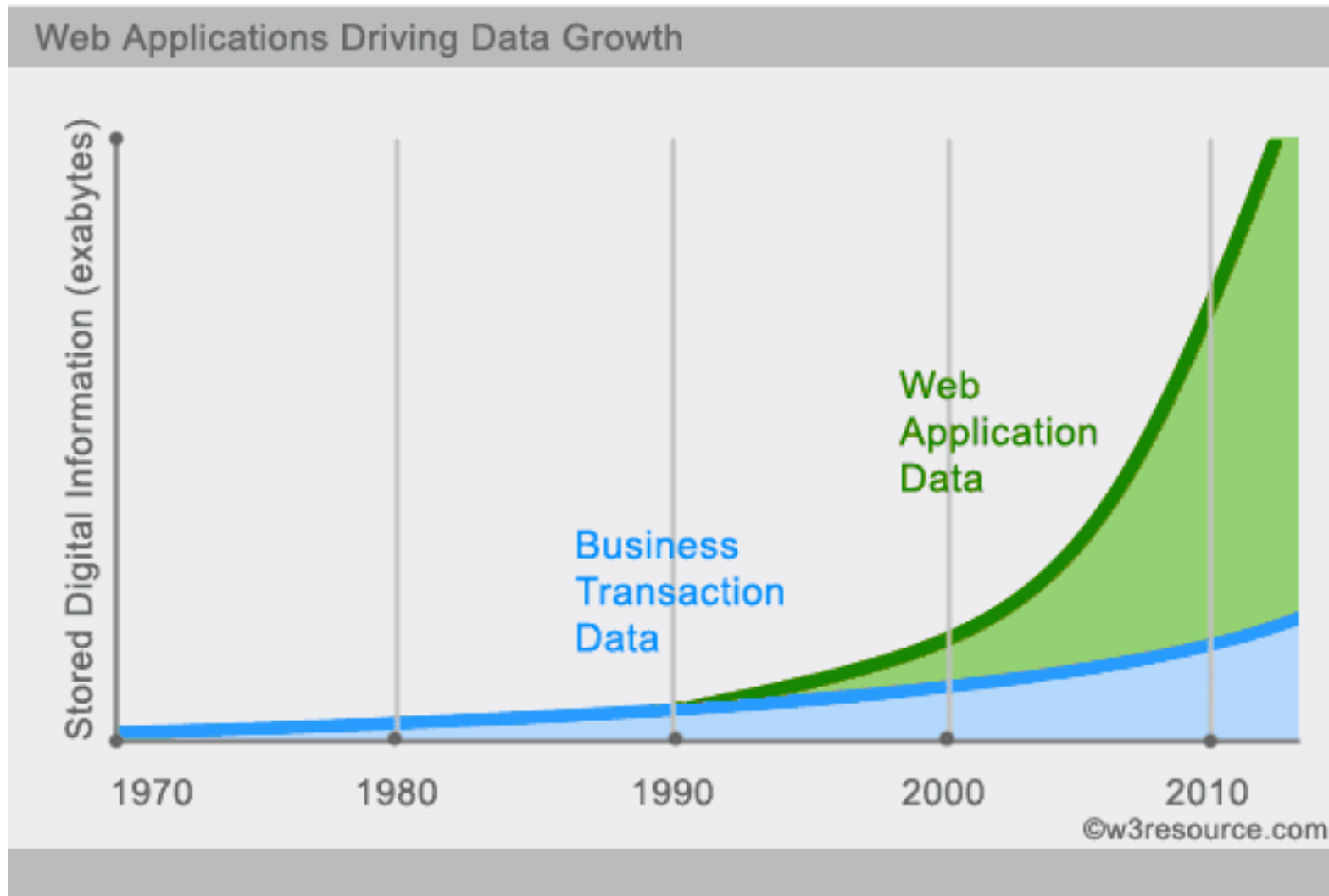
Who uses NoSQL?

Google

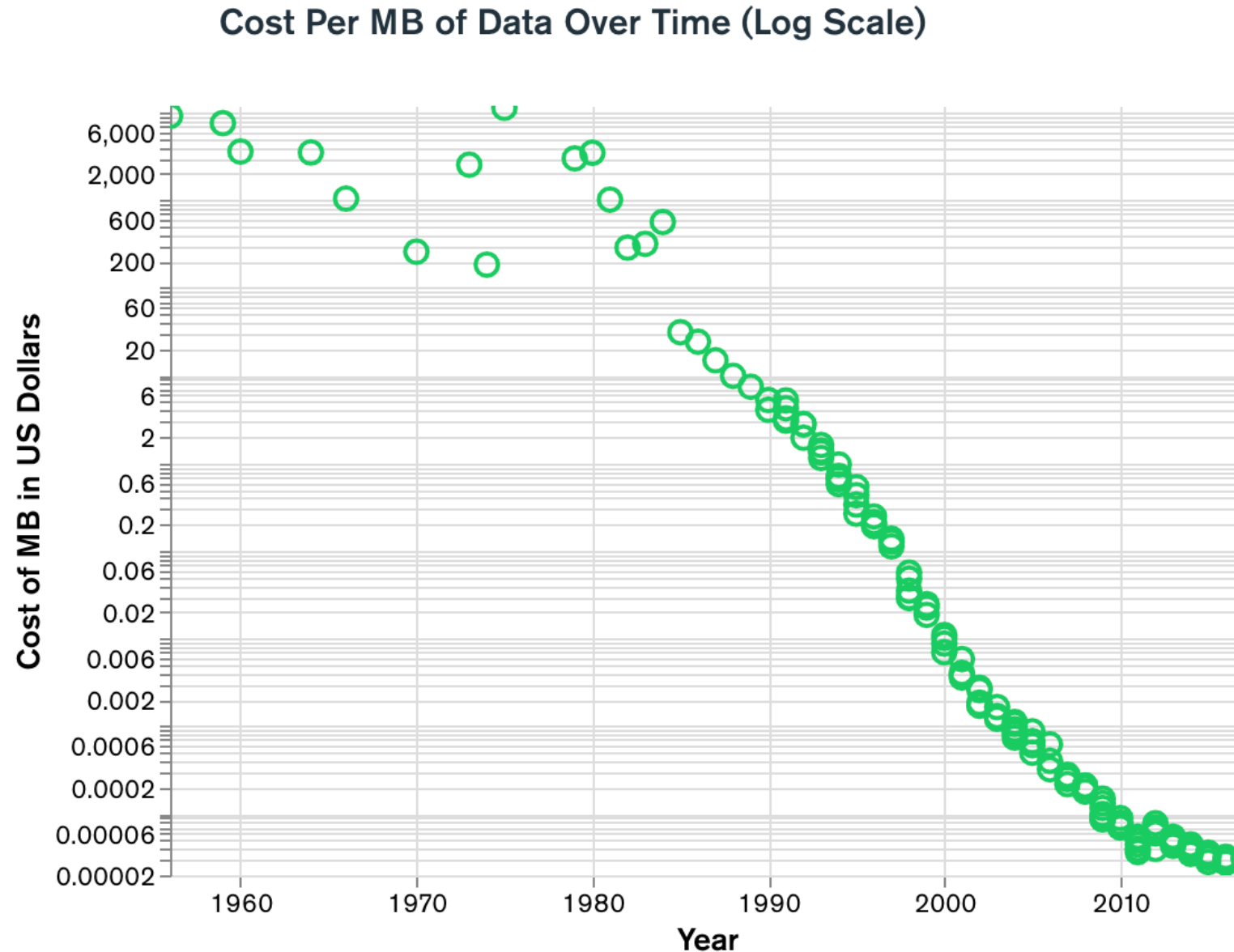
LinkedIn



Why NoSQL: Growth in Web Applications



Why NoSQL: Volume, Velocity



A Brief Overview of NoSQL Systems

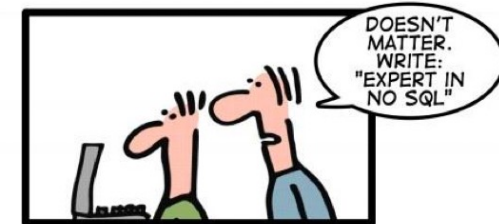
1. Horizontally scalability
 2. Replicate/distribute data over many servers
 3. Simple call interface
 4. Often weaker concurrency model than RDBMS
 5. Efficient use of distributed indexes and RAM
 6. Flexible schemas
- } Volume, Velocity
- } Variety

NoSQL and Distributed Databases 1

1. What is NoSQL?

2. Major types of NoSQL systems

HOW TO WRITE A CV



Leverage the NoSQL boom

(Major) Types of NoSQL databases

- Key-value stores



- Wide column databases



- Document stores



- Graph databases



- Vector databases



Key-Value Stores



Key-Value Stores: Data Model

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

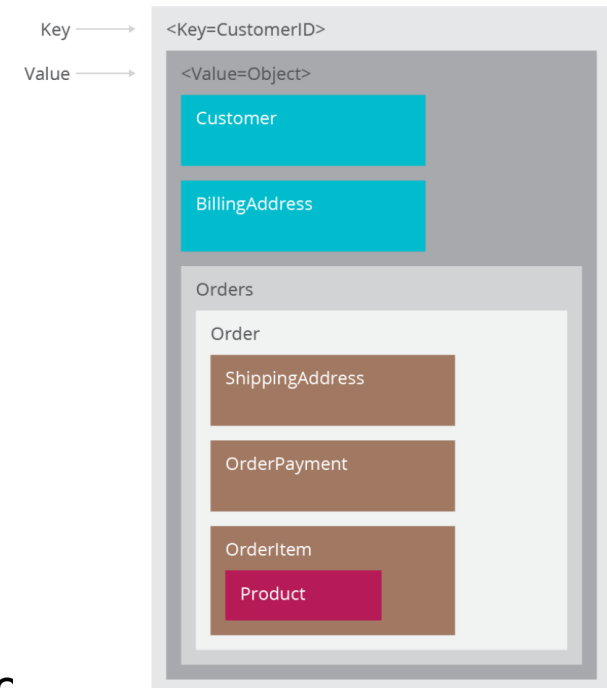
MAC table

Key	Value
10.94.214.172	3c:22:fb:86:c1:b1
10.94.214.173	00:0a:95:9d:68:16
10.94.214.174	3c:1b:fb:45:c4:b1

- Stores associations between keys and values
 - Based on Amazon's Dynamo paper (2007)
- Keys are usually primitives and can be queried
 - For example, ints, strings, raw bytes, etc.
- Values can be primitive or complex; usually cannot be queried
 - Examples: ints, strings, lists, JSON, HTML fragments, BLOB (basic large object), etc.

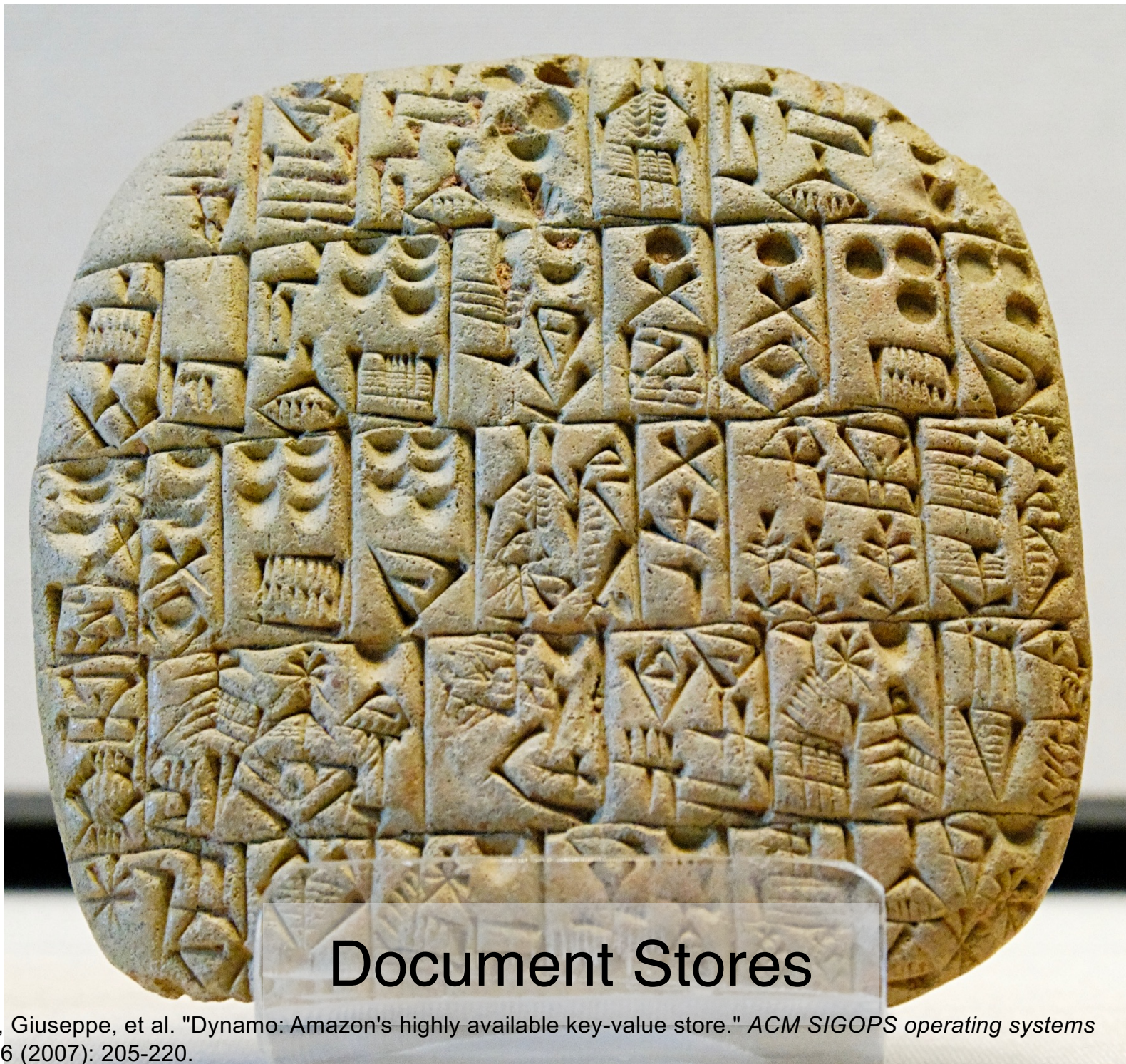
Key-Value Stores: Operations

- Very simple API:
 - Get – fetch value associated with key
 - Put – set value associated with key
- Optional operations:
 - Multi-get
 - Multi-put
 - Range queries
- Suitable for:
 - Small continuous read and writes
 - Storing 'basic' information (e.g. raw chunks of byte)
 - When complex queries are not required / rarely required
- Example Applications
 - Storing user sessions
 - Caches
 - User data that is often processed individually: e.g. patient medical data?
 - Only if no queries like 'How many patients who took X treatment recovered?'



Key-Value Stores: Implementation

- Non-persistent:
 - Just a big in-memory hash table
 - Examples: Memcached, Redis
 - (But: these can also back up the data to disk periodically)
- Persistent
 - Data is stored persistently to disk
 - Examples: RocksDB, Dynamo, Riak



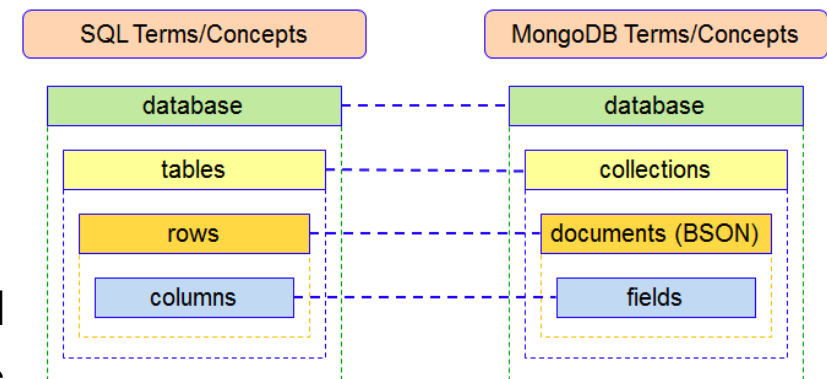
Document Stores

DeCandia, Giuseppe, et al. "Dynamo: Amazon's highly available key-value store." *ACM SIGOPS operating systems review* 41.6 (2007): 205-220.

Document Stores: Document Model



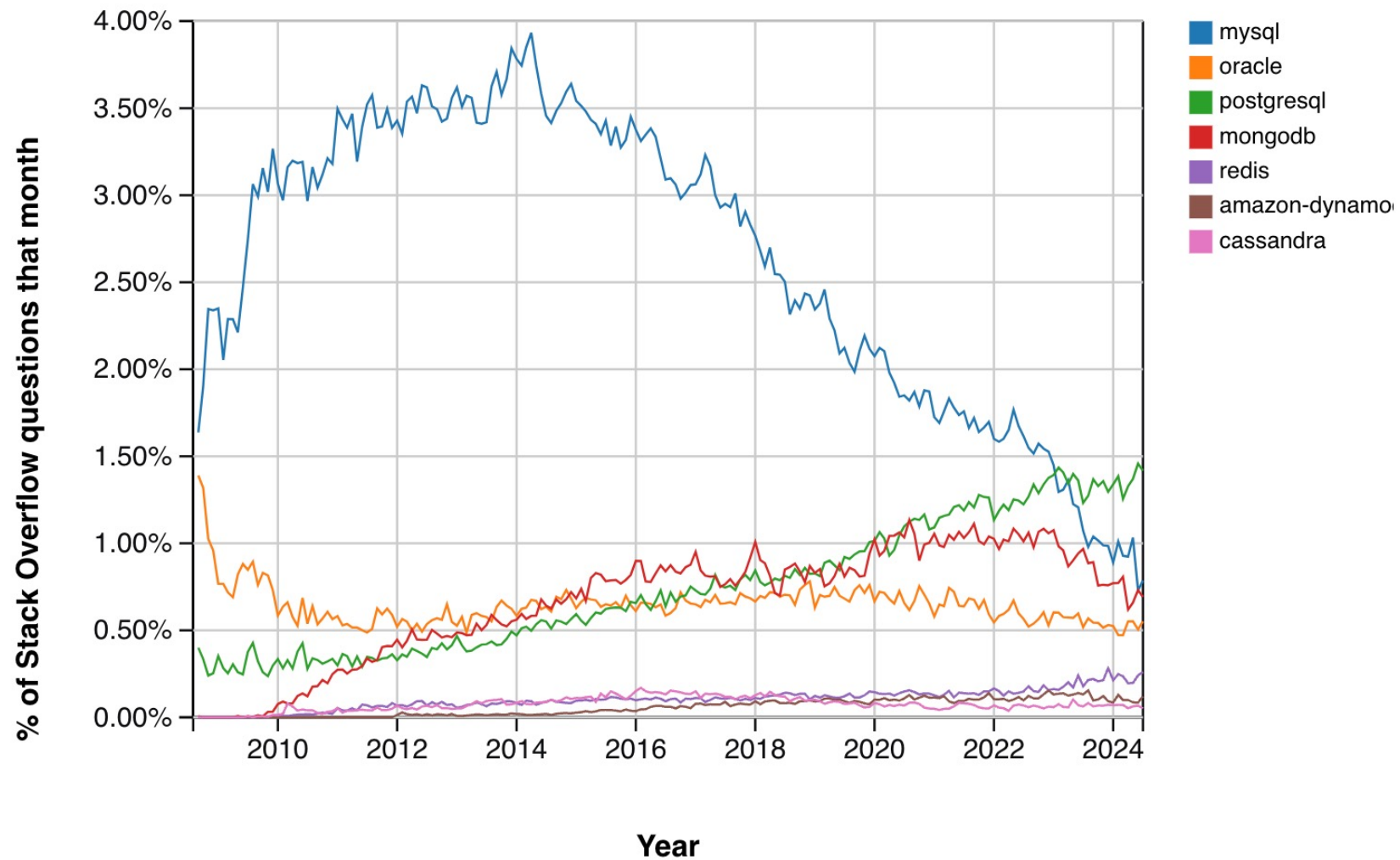
- A database can have multiple collections
- Collections have multiple documents
- A document is a JSON-like object: it has fields and values
 - Different documents can have different field
 - Can be nested: i.e. JSON objects as values



Popularity of DBMS

Rank			DBMS	Database Model	Score		
Sep 2023	Aug 2023	Sep 2022			Sep 2023	Aug 2023	Sep 2022
1.	1.	1.	Oracle +	Relational, Multi-model i	1240.88	-1.22	+2.62
2.	2.	2.	MySQL +	Relational, Multi-model i	1111.49	-18.97	-100.98
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model i	902.22	-18.60	-24.08
4.	4.	4.	PostgreSQL +	Relational, Multi-model i	620.75	+0.37	+0.29
5.	5.	5.	MongoDB +	Document, Multi-model i	439.42	+4.93	-50.21
6.	6.	6.	Redis +	Key-value, Multi-model i	163.68	+0.72	-17.79
7.	7.	7.	Elasticsearch	Search engine, Multi-model i	138.98	-0.94	-12.46
8.	8.	8.	IBM Db2	Relational, Multi-model i	136.72	-2.52	-14.67
9.	↑ 10.	↑ 10.	SQLite +	Relational	129.20	-0.72	-9.62
10.	↓ 9.	↓ 9.	Microsoft Access	Relational	128.56	-1.78	-11.47
11.	11.	↑ 13.	Snowflake +	Relational	120.89	+0.27	+17.39
12.	12.	↓ 11.	Cassandra +	Wide column, Multi-model i	110.06	+2.67	-9.06
13.	13.	↓ 12.	MariaDB +	Relational, Multi-model i	100.45	+1.80	-9.70
14.	14.	14.	Splunk	Search engine	91.40	+2.42	-2.65
15.	↑ 16.	↑ 16.	Microsoft Azure SQL Database	Relational, Multi-model i	82.73	+3.22	-1.69
16.	↓ 15.	↓ 15.	Amazon DynamoDB +	Multi-model i	80.91	-2.64	-6.51
17.	↑ 18.	↑ 20.	Databricks	Multi-model i	75.18	+3.84	+19.56
18.	↓ 17.	↓ 17.	Hive	Relational	71.83	-1.52	-6.60
19.	19.	↓ 18.	Teradata	Relational, Multi-model i	60.33	-0.98	-6.25
20.	20.	↑ 24.	Google BigQuery +	Relational	56.46	+2.56	+6.34

Stack Overflow Trends



Document Stores: Querying

- Unlike (basic) key value stores, document stores allow some querying based on the content of a document
- CRUD = Create, Read, Update, Delete

[Optional] CRUD: Create

In MongoDB

```
db.users.insert (  ← collection
  {
    name: "sue",    ← field: value
    age: 26,        ← field: value
    status: "A"     ← field: value
  }                } document
)
```

In SQL

```
INSERT INTO users      ← table
      ( name, age, status ) ← columns
VALUES      ( "sue", 26, "A" ) ← values/row
```

[Optional] CRUD: Read

In MongoDB

```
db.users.find(  
  { age: { $gt: 18 } },  
  { name: 1, address: 1 }  
) .limit(5)
```

← collection
← query criteria
← projection
← cursor modifier

In SQL

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

[Optional] CRUD: Update

In MongoDB

```
db.users.update(  
  { age: { $gt: 18 } },  
  { $set: { status: "A" } },  
  { multi: true }  
)
```

← collection
← update criteria
← update action
← update option

In SQL

```
UPDATE users  
SET status = 'A'  
WHERE age > 18
```

← table
← update action
← update criteria

[Optional] CRUD: Delete

In MongoDB

```
db.users.remove(  
  { status: "D" }  
)
```

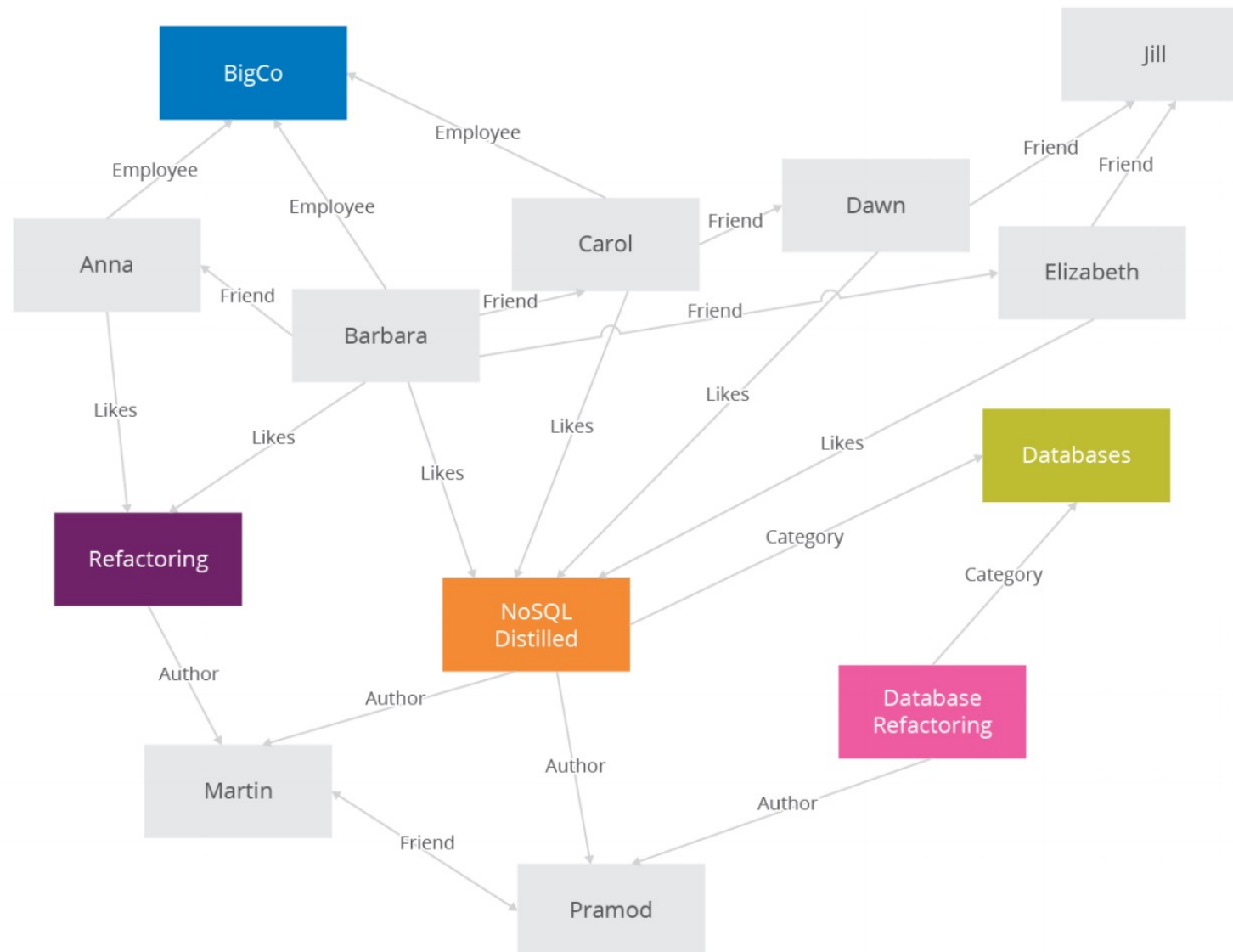
← collection
← remove criteria

In SQL

```
DELETE FROM users  
WHERE status = 'D'
```

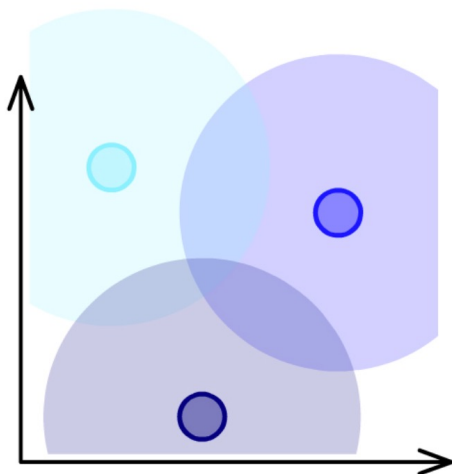
← table
← delete criteria

Graph Databases



Vector Databases

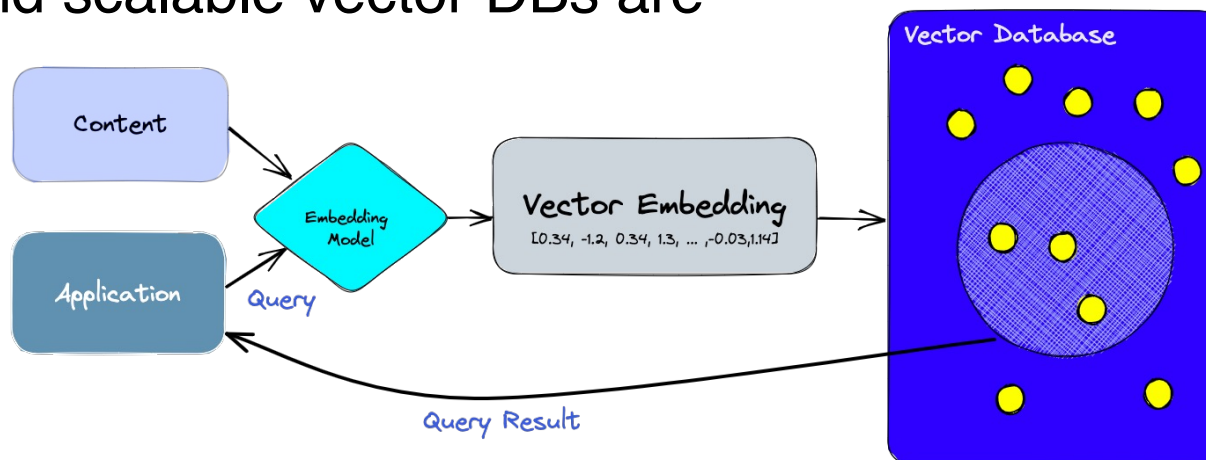
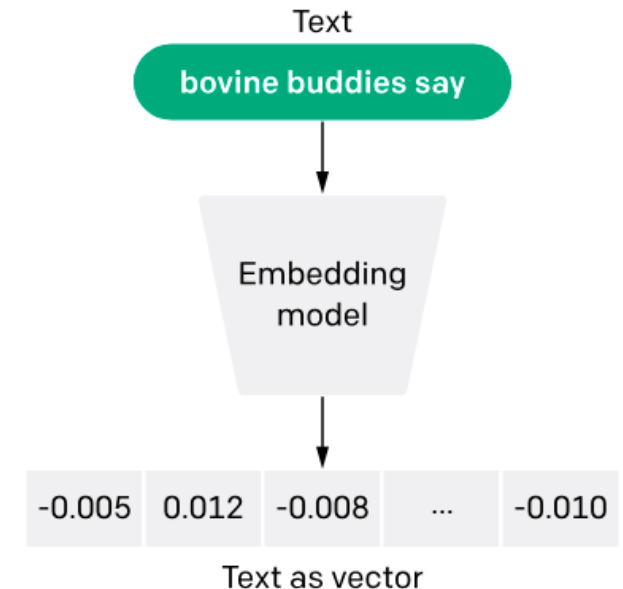
- Store vectors (i.e., each row represents a point in d dimensions)
 - Usually dense, numerical, and high-dimensional
- Allow fast similarity search, i.e., given a query, retrieve similar neighbors from the database.
 - Earlier in class we discussed similarity search using min-hashing. One of the major types of algorithms used in vector databases is locality-sensitive hashing (LSH), which is closely related to min-hashing.
- DB features: scalability, real-time updates, replication
- Examples: Milvus, Redis, Weaviate, MongoDB Atlas



```
>>> collection = Collection(name='my_collection', data=None, schema=schema)
>>> index_params = {
    'index_type': 'IVF_FLAT',
    'params': {'nlist': 1024},
    'metric_type': 'L2'
}
>>> collection.create_index('embedding', index_params)
>>> search_param = {
    'data': vector,
    'anns_field': 'embedding',
    'param': {'metric_type': 'L2', 'params': {'nprobe': 16}},
    'limit': 10,
    'expr': 'id_field > 0'
}
>>> results = collection.search(**search_param)
```

Vector Databases in AI / ML

- Vector DBs have become a hot topic recently due to AI / ML
 - Large language models are often used to convert some text into vectors (or embeddings) useful for applications like search, recommendation, clustering
 - Similarly, vision models convert images into embeddings
- To quickly search for similar embeddings (e.g., for search / recommendation), efficient and scalable vector DBs are needed



Conclusion: Pros & Cons of NoSQL Systems

Conclusion: depends on needs of application: whether denormalization is suitable; complexity of queries (joins vs simple read/writes); importance of consistency (e.g. financial transactions vs tweets); data volume / need for availability.

- Binary view is oversimplified: NoSQL databases often still have SQL-like query languages (e.g. Cassandra) and tunable consistency levels.
- Other alternative solutions: “NewSQL”, distributed SQL (e.g. Presto), PostgreSQL JSON, ...
- Need to understand tradeoffs to make an informed decision!

Q1: True or false: an advantage of document stores over key-value stores is it is easier to store unstructured objects.

1. True
2. False

Q1: True or false: an advantage of document stores over key-value stores is it is easier to store unstructured objects.

1. True

2. False

Q2: True or false: an advantage of document stores over key-value stores is it is easier to query for all documents fitting a condition.

1. True
2. False

Q2: True or false: an advantage of document stores over key-value stores is it is easier to query for all documents fitting a condition.

1. True

2. False

Take-away in the AI Era

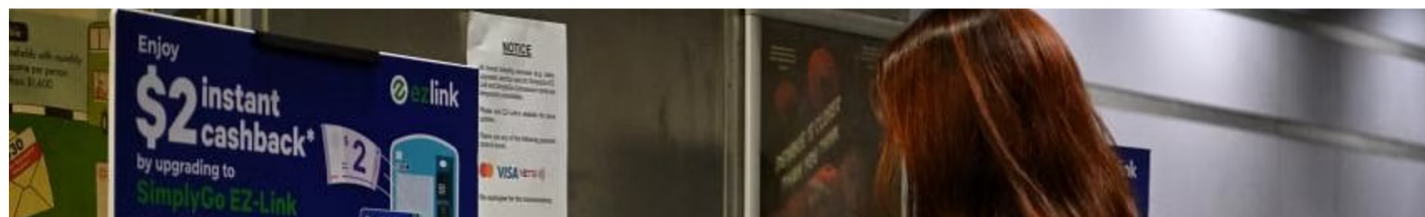
- (Why big database design decisions still require humans)
-  What AI cannot decide for you (yet)
 - Which **consistency model** your application can tolerate
 - When **eventual consistency** is acceptable — and when it is dangerous
 - How data model choices affect **correctness, cost, and user experience**
- **Your design matters (AI can assist)**
 - Reasoning about consistency vs availability
 - Designing data models around access patterns
 - Handling conflicts and anomalies
 - Understanding the cost of “convenience”

Acknowledgements

- CS4225 slides by Bryan Hooi
- Penn State CMPSC 431W Database Management Systems (Fall 2015) – McGrawHill, Wang-Chien Lee, Yu-San Lin
- mongodb.com, redis.com, openai.com, pinecone.io
- Otter video from https://www.reddit.com/r/singapore/comments/mezgg3/otters_on_the_grass_at_botanic_gardens_today/
- SimplyGo: https://www.reddit.com/r/singapore/comments/196e2v8/i_try_to_illustrate_the_technical_difficulties_in/
- Cat: https://www.reddit.com/r/nus/comments/tyaxm3/so_cute_so_chiil_so_cat/

Supplementary Slides

SimplyGo to replace older payment cards for adult public transport fares from June 1



Do you need to change your fare card?

From June, Nets FlashPay will not be accepted on public transport. Older ez-link adult cards that are not on SimplyGo must be upgraded.

ST

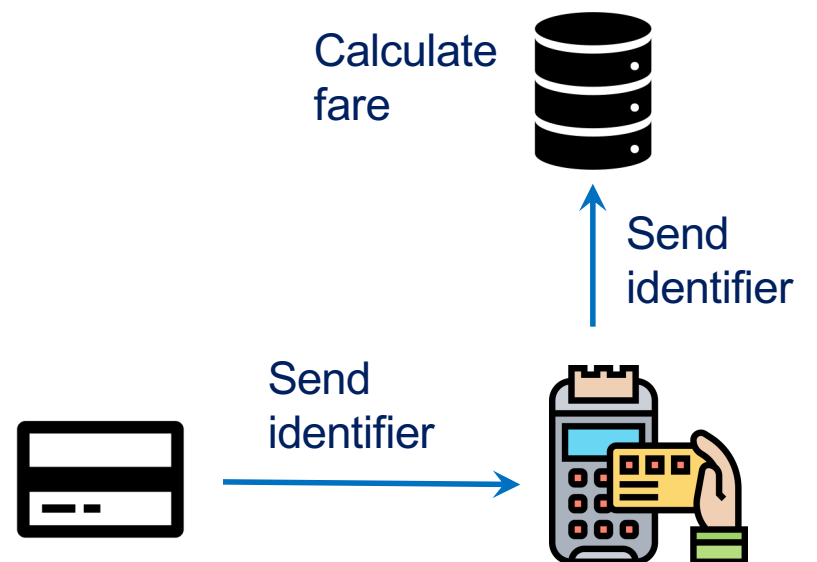
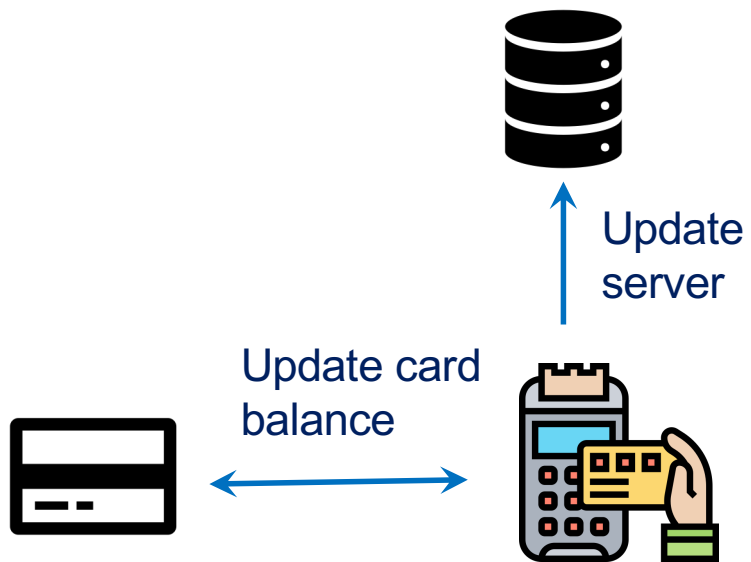


askST: Why can't I see fares charged on card readers with SimplyGo?



LTA shelves plan to replace older public transport payment cards with SimplyGo by June

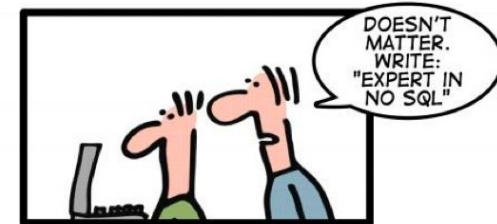




Key concepts in NoSQL

- Eventual Consistency
- Duplication

HOW TO WRITE A CV

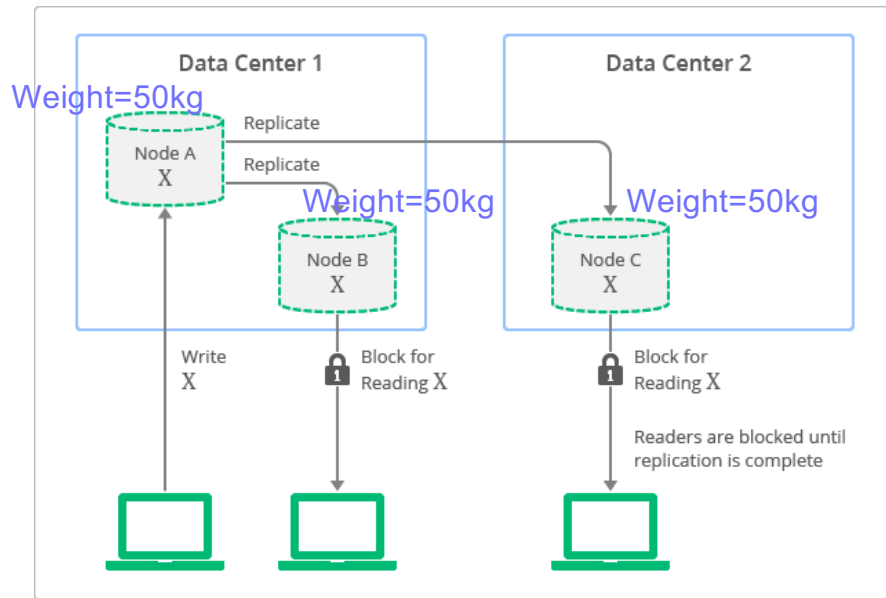


Leverage the NoSQL boom

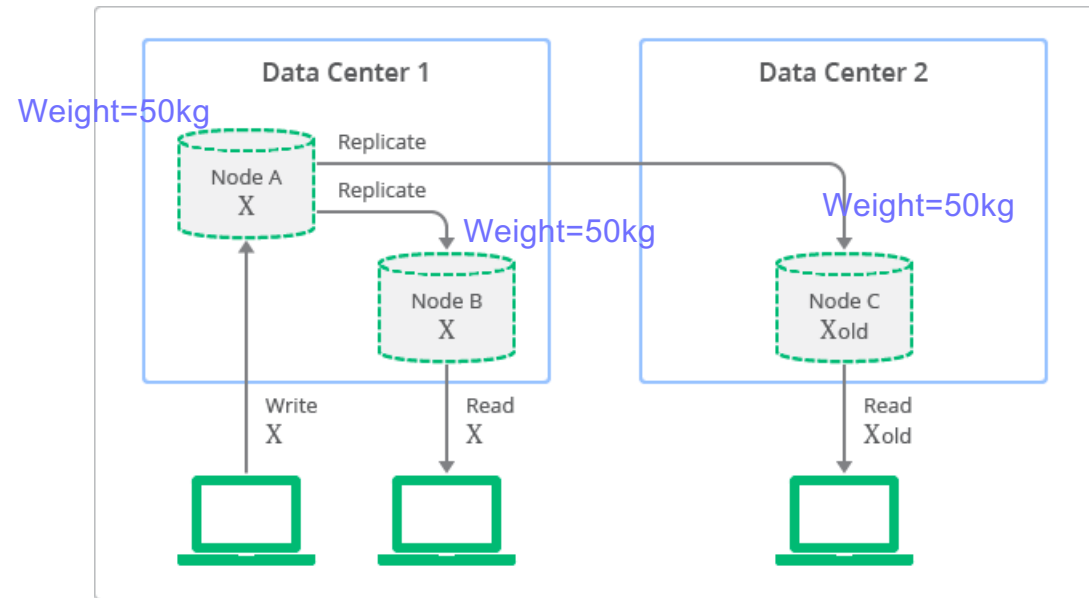
Clarification: Strong Consistency vs ACID

- If you've taken a database class, you may recall that the C in ACID stands for "Consistency"
 - This is almost completely unrelated with our "Strong Consistency"!
 - "Consistency" in ACID: means the database must always obey certain predefined constraints (e.g. inventory counts cannot be negative)
 - Strong Consistency (in distributed systems): any reads immediately after an update give the same result on all observers
 - The names came from different communities (ACID from the database community; Strong Consistency from distributed systems)
 - Note: since we didn't discuss ACID much in this class, we consider it as not required knowledge for exam purposes

Strong vs Eventual Consistency

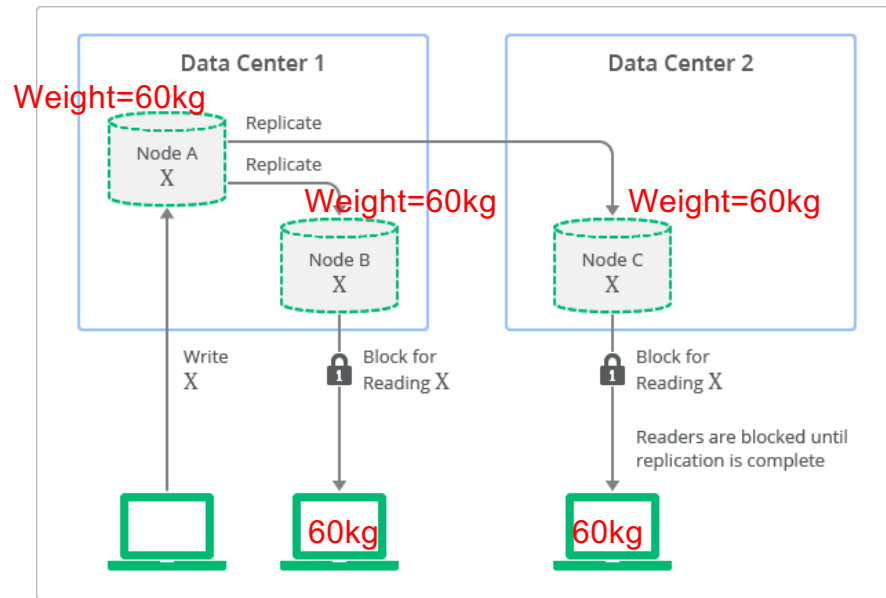


Strong consistency: any reads immediately after an update must give the same result on all observers



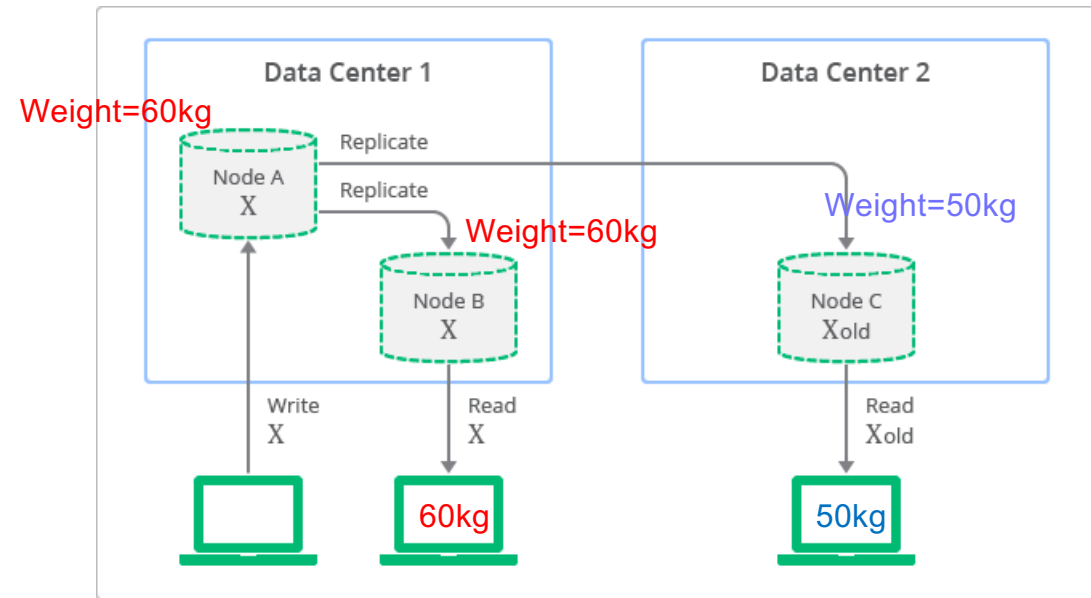
Eventual consistency: if the system is functioning and we wait long enough, eventually all reads will return the last written value

Strong vs Eventual Consistency



All readers read new value (60kg)

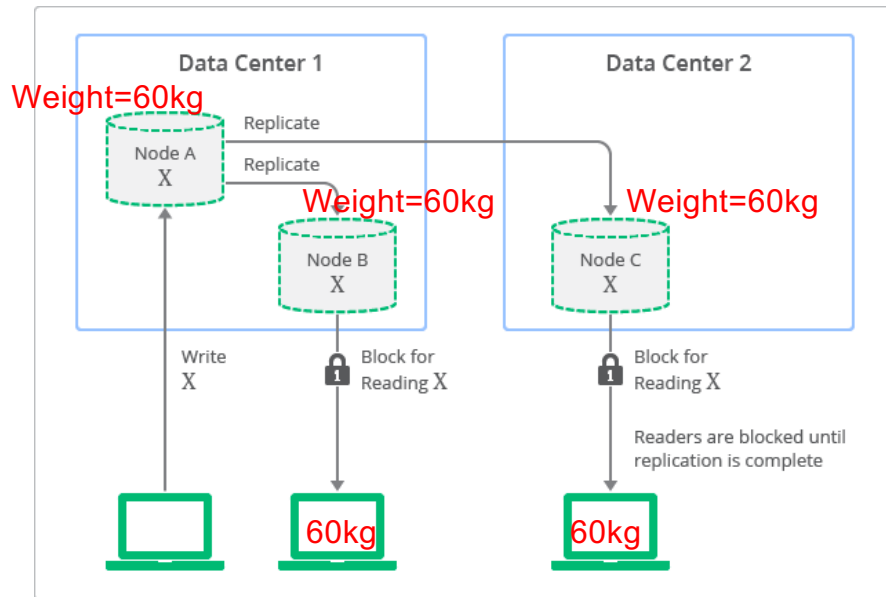
Strong consistency: any reads immediately after an update must give the same result on all observers



Readers may read old value (50kg)

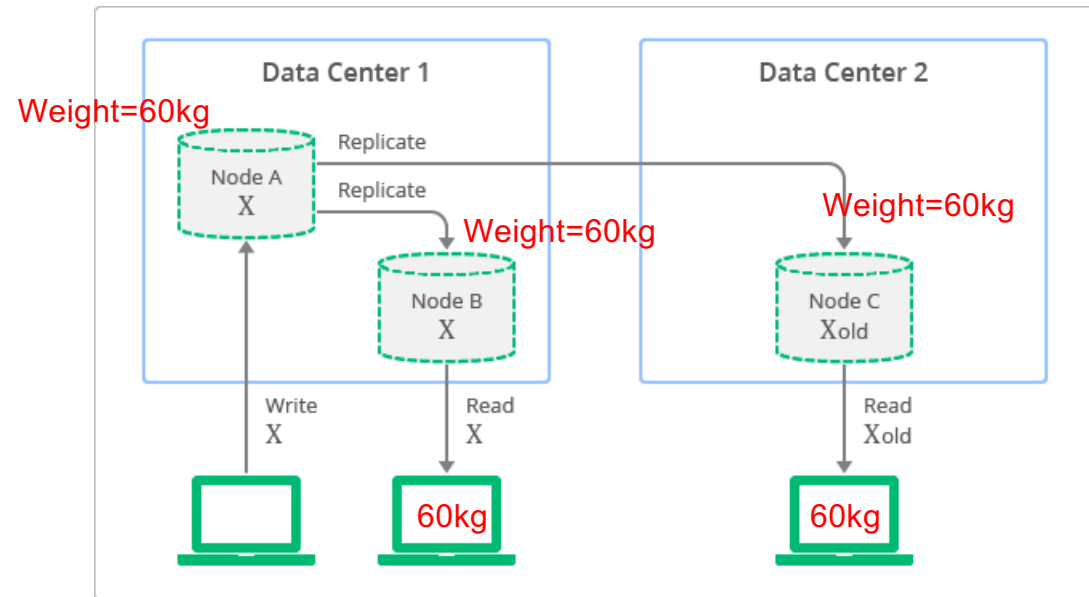
Eventual consistency: if the system is functioning and we wait long enough, eventually all reads will return the last written value

Strong vs Eventual Consistency



All readers read new value (60kg)

Strong consistency: any reads immediately after an update must give the same result on all observers



Eventually, the update is propagated to Node C; after which, readers will read the correct value

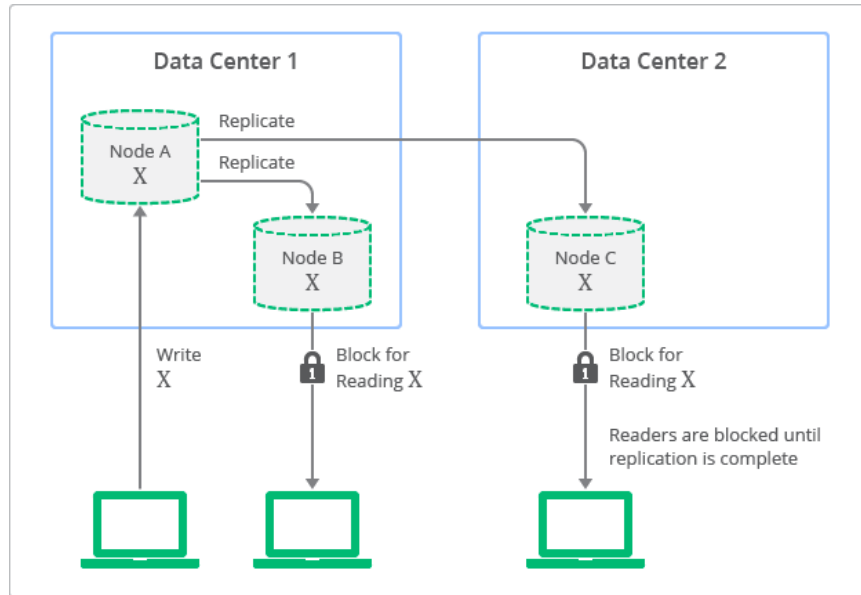
Eventual consistency: if the system is functioning and we wait long enough, eventually all reads will return the last written value

Eventual Consistency

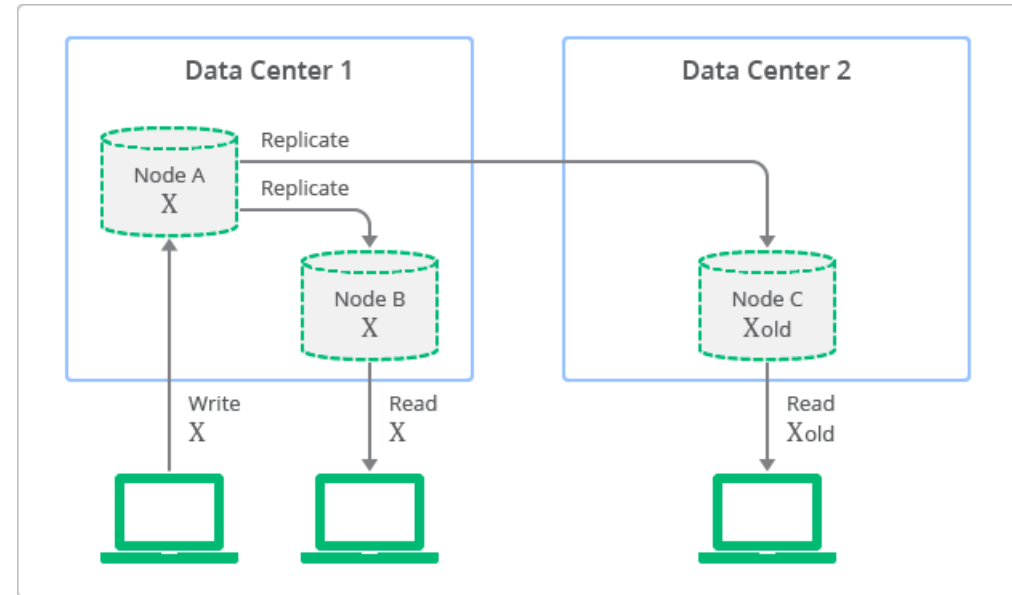
- ACID vs BASE: Relational DBMS provide stronger (ACID) guarantees, but many NoSQL system relax this to weaker “BASE” approach:
 - Basically Available: basic reading and writing operations are available most of the time
 - Soft State: without guarantees, we only have some probability of knowing the state at any time
 - Eventually Consistent: Contrast to “strong consistency”:

Strong vs Eventual Consistency

Strong consistency

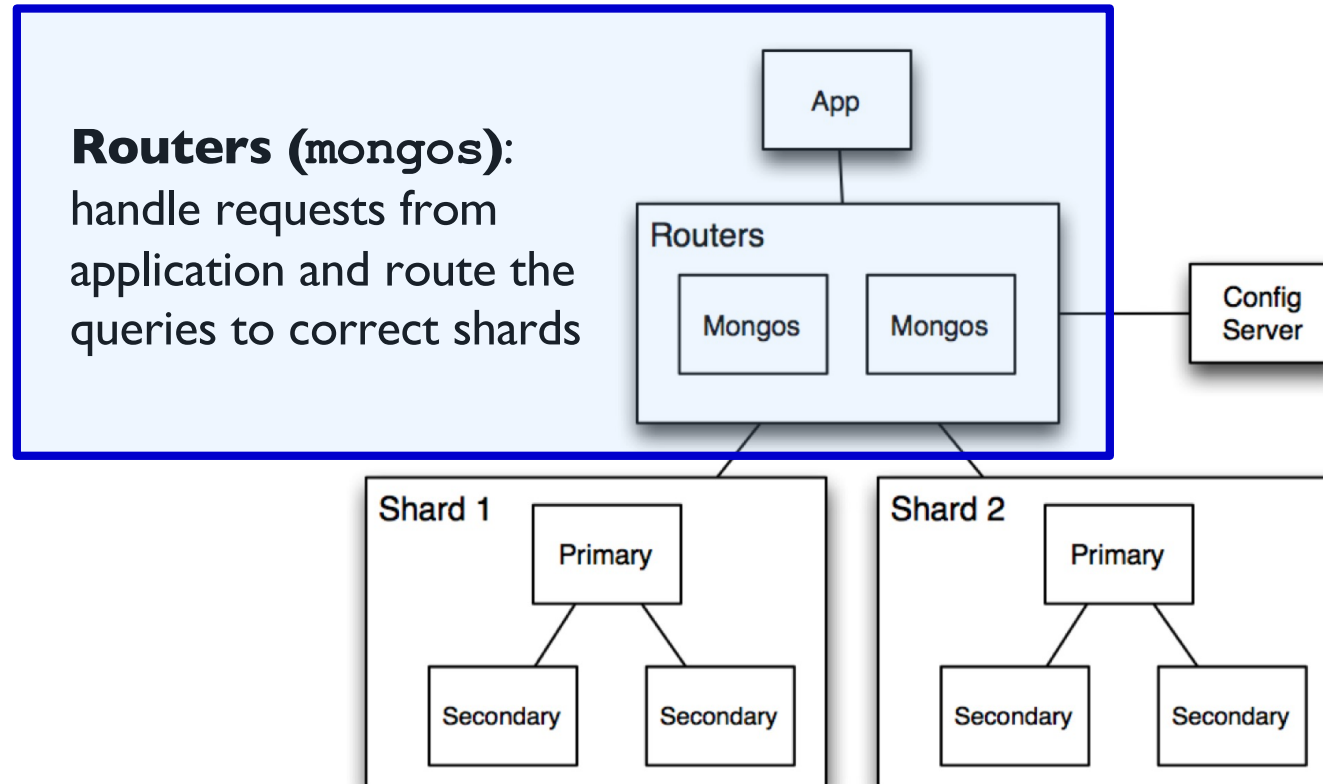


Eventual consistency



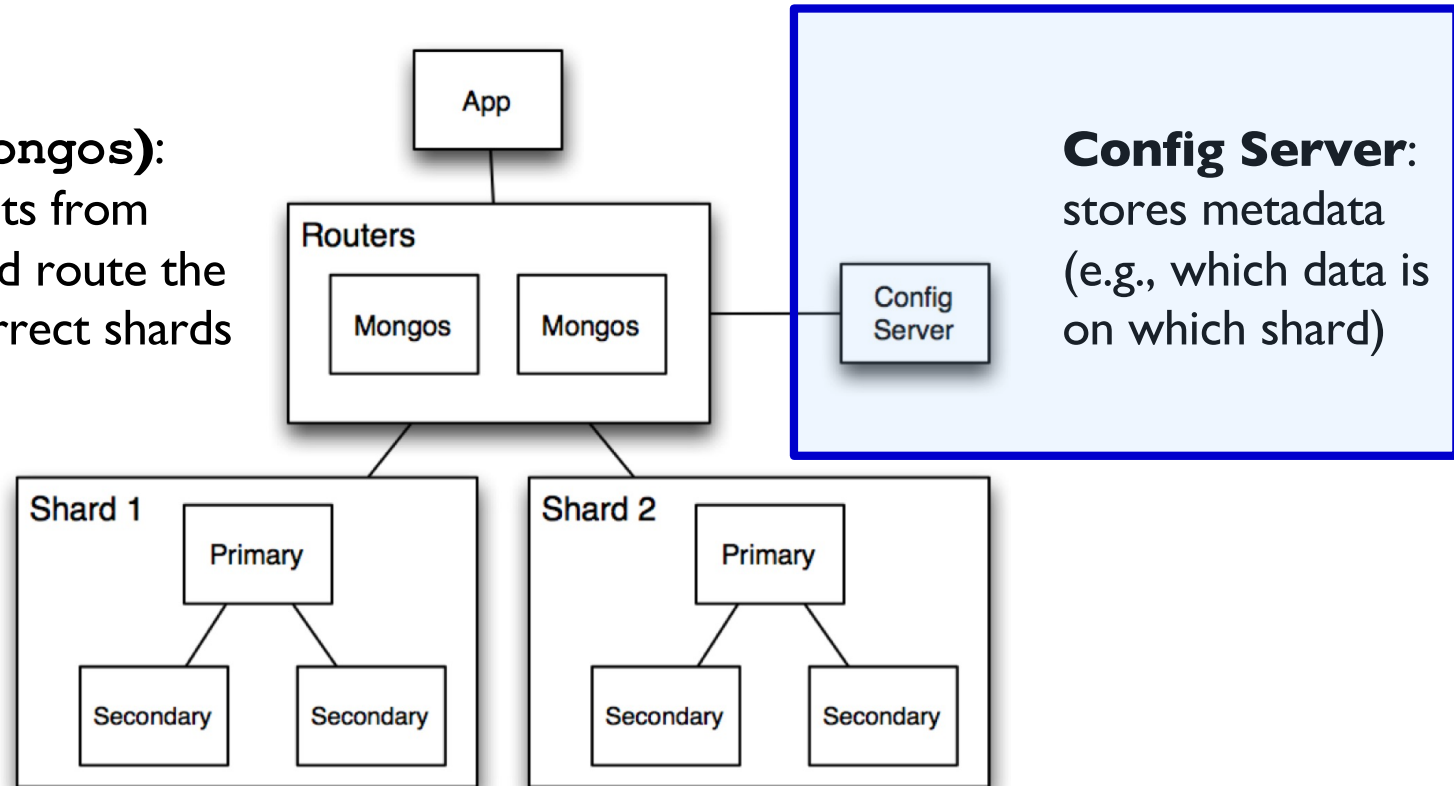
- **Implications:** eventual consistency offers better availability at the cost of a much weaker consistency guarantee. This may be acceptable for some applications (e.g. statistical queries, tweets, social network feed, ...) but not for others (e.g. financial transactions)
 - Note that while NoSQL systems allow for weaker consistency guarantees, many more recent systems / versions are often configurable, i.e. can be configured for multiple different consistency levels (including strong) – ‘tunable consistency’

Architecture of MongoDB



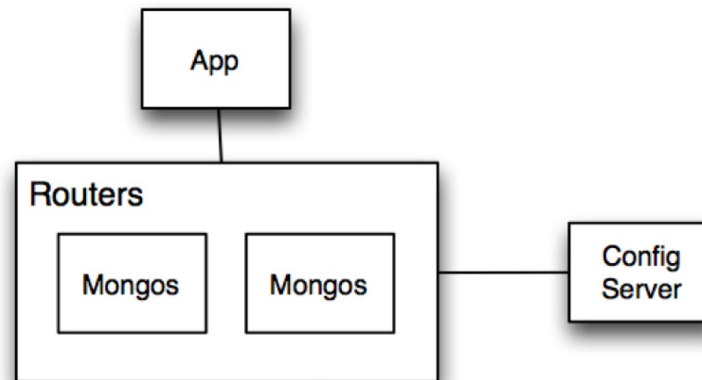
Architecture of MongoDB

Routers (mongos):
handle requests from application and route the queries to correct shards



Architecture of MongoDB

Routers (mongos):
handle requests from application and route the queries to correct shards

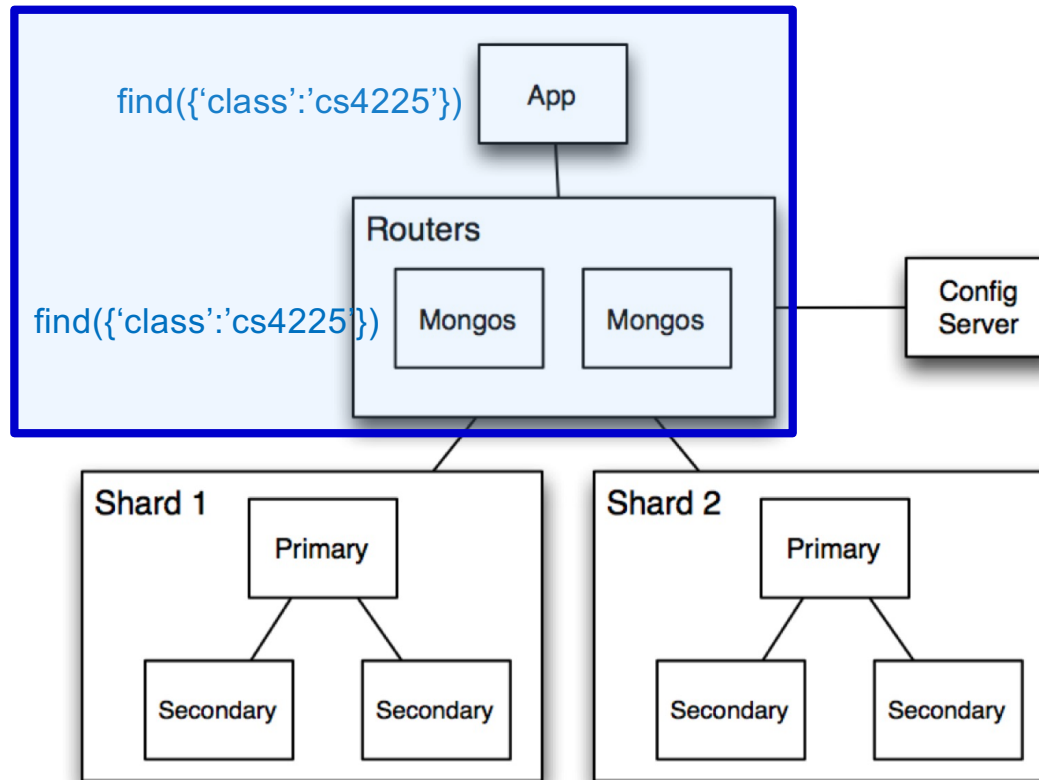


Config Server:
stores metadata
(e.g., which data is on which shard)

Replica sets: multiple processes which maintain the same data, for redundancy

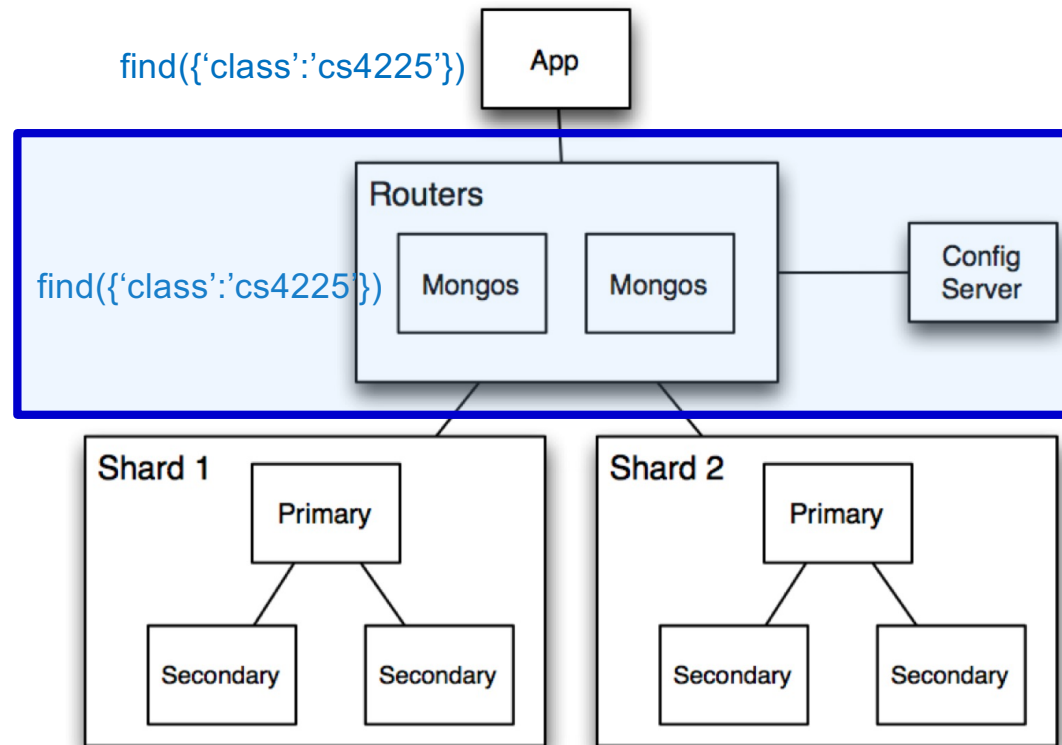
Each replica set is responsible for a subset of the database, known as a **shard** (this is a **horizontal partitioning** scheme)

Example of Read or Write Query



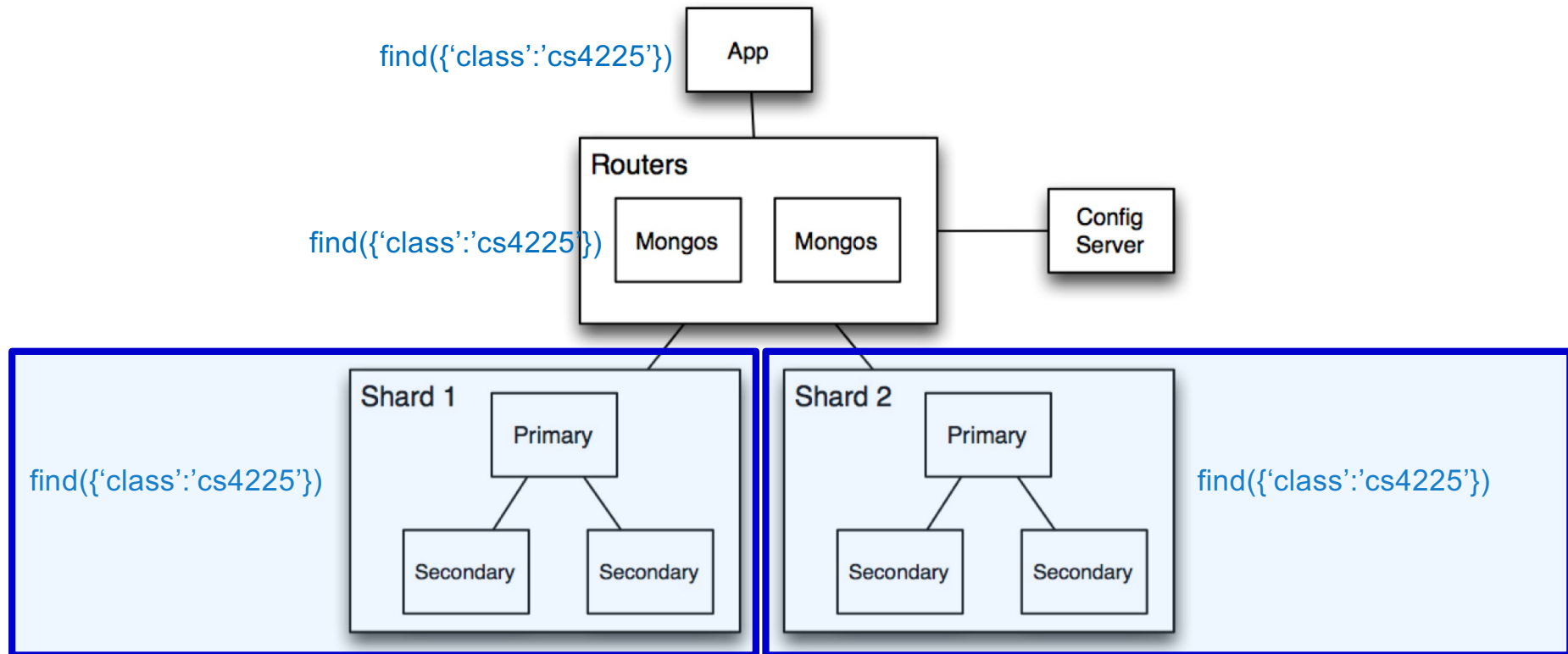
1. Query is issued to a router (**mongos**) instance

Example of Read or Write Query



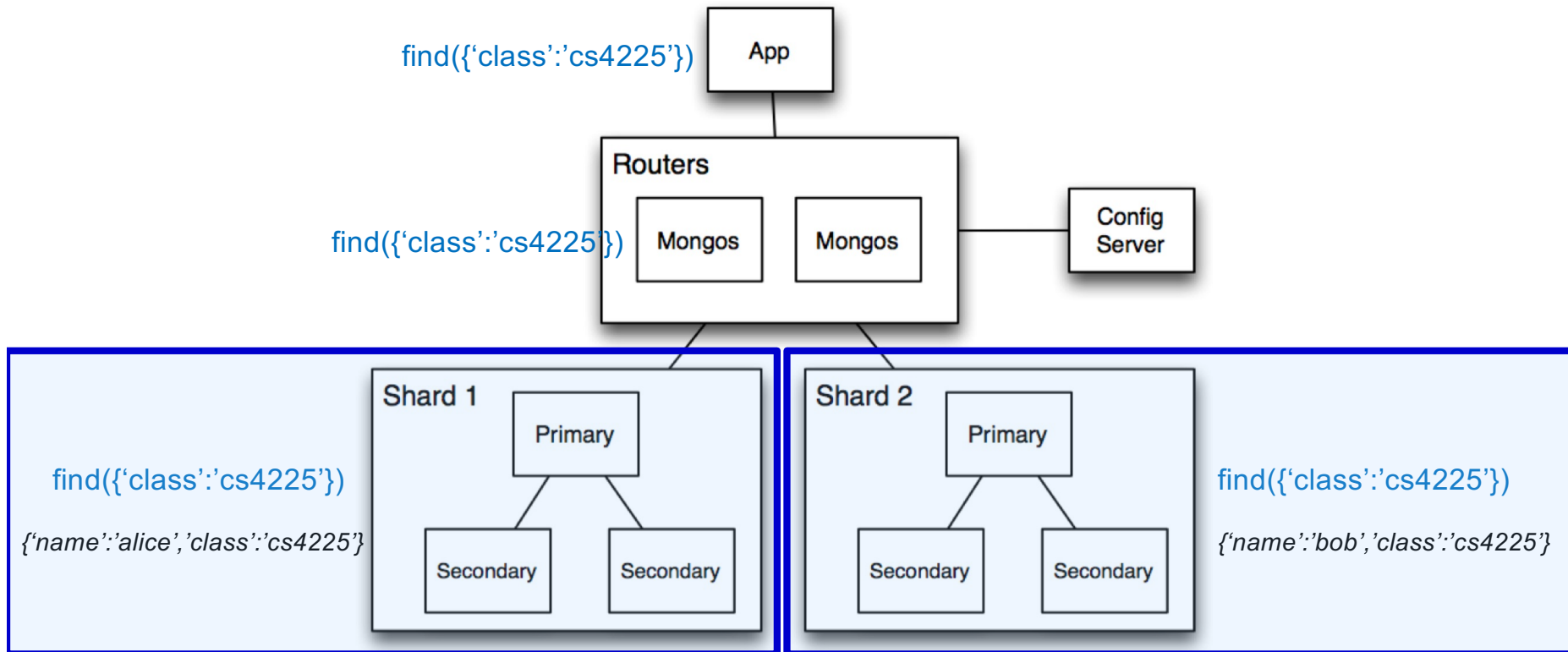
1. Query is issued to a router (`mongos`) instance
2. **With help of config server, `mongos` determines which shards to query**

Example of Read or Write Query



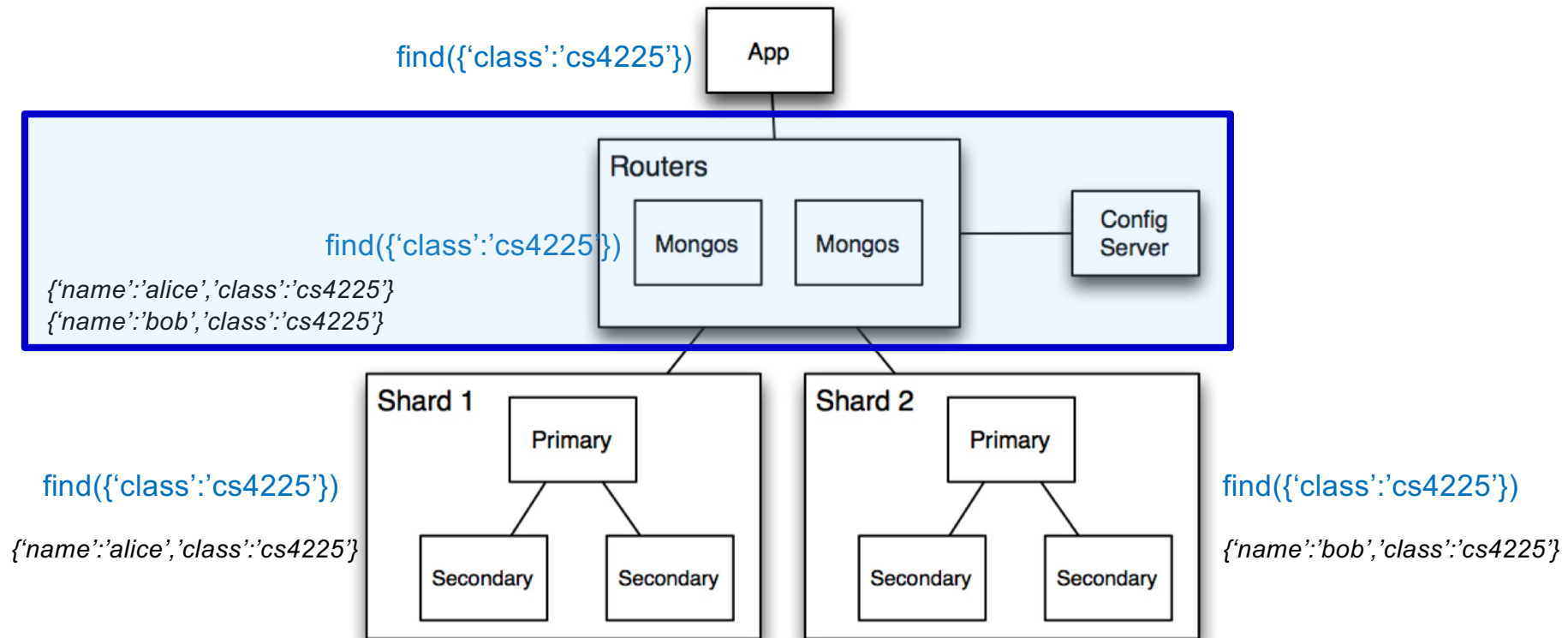
1. Query is issued to a router (`mongos`) instance
2. With help of config server, `mongos` determines which shards to query
3. **Query is sent to relevant shards (*partition pruning*)**
 - Example: when reading a specific value of the shard key, the config server can determine that the query only need to go to one shard (the one that contains that value of the shard key); writes are similar
 - But if the query is based on a key other than the shard key, it is relevant to all shards, and thus will go to all shards

Example of Read or Write Query



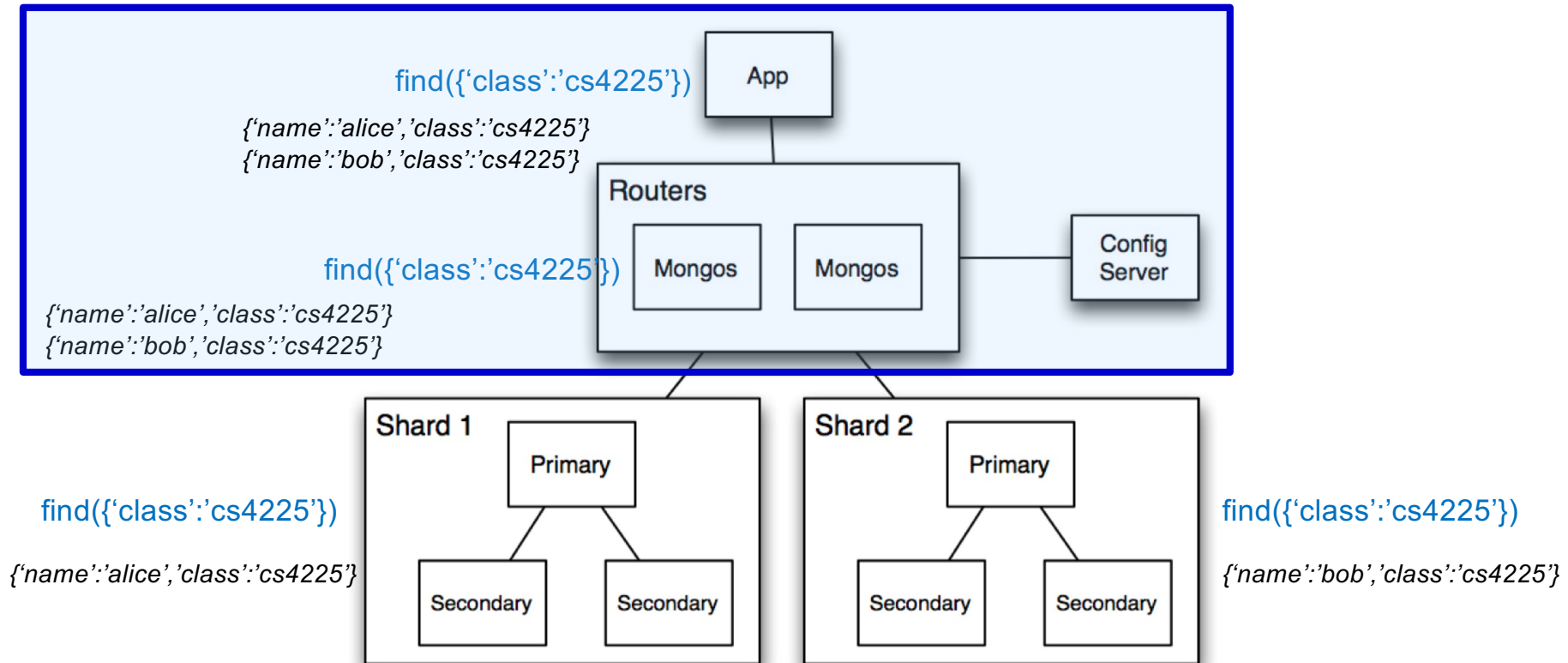
1. Query is issued to a router (`mongos`) instance
2. With help of config server, `mongos` determines which shards to query
3. Query is sent to relevant shards (*partition pruning*)
4. **Shards run query on their data, and send results back to `mongos`**

Example of Read or Write Query



1. Query is issued to a router (`mongos`) instance
2. With help of config server, `mongos` determines which shards to query
3. Query is sent to relevant shards (*partition pruning*)
4. Shards run query on their data, and send results back to `mongos`
5. `mongos` merges the query results and returns the merged results to the application

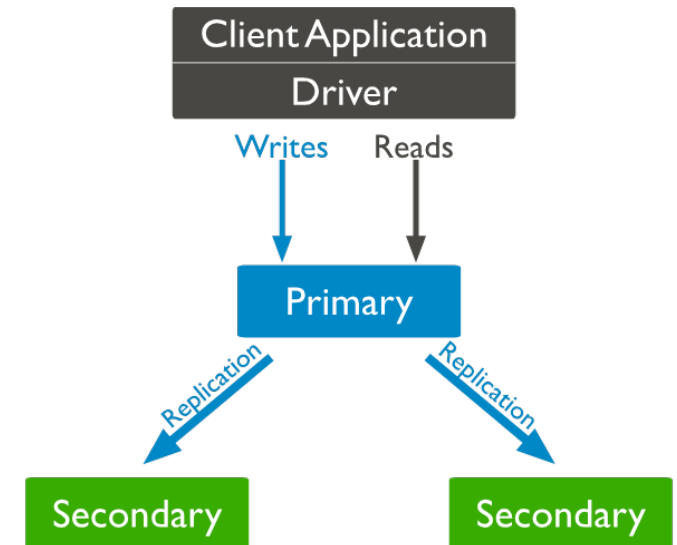
Example of Read or Write Query



1. Query is issued to a router (`mongos`) instance
2. With help of config server, `mongos` determines which shards to query
3. Query is sent to relevant shards (*partition pruning*)
4. Shards run query on their data, and send results back to `mongos`
5. `mongos` merges the query results and returns the merged results to the application

Replication in MongoDB

- Standard configuration: 1 primary, 2 secondaries
- **Writes:**
 - The “Primary” receives all write operations
 - Records writes onto its “operation log”
 - Secondaries will then replicate this operation log, apply it to their local copies of the data (thus ensuring data is synchronized), then acknowledge the operation
- **Reads:**
 - The user can configure the “read preference”, which decides whether we can read from secondaries (this is the default), or the primary
 - Allowing reading from secondaries can decrease latency and distribute load (improving throughput), but allows for reading stale data (it has only “eventual consistency”)



Q3: True or false: in MongoDB, shards are beneficial to allow for redundancy.

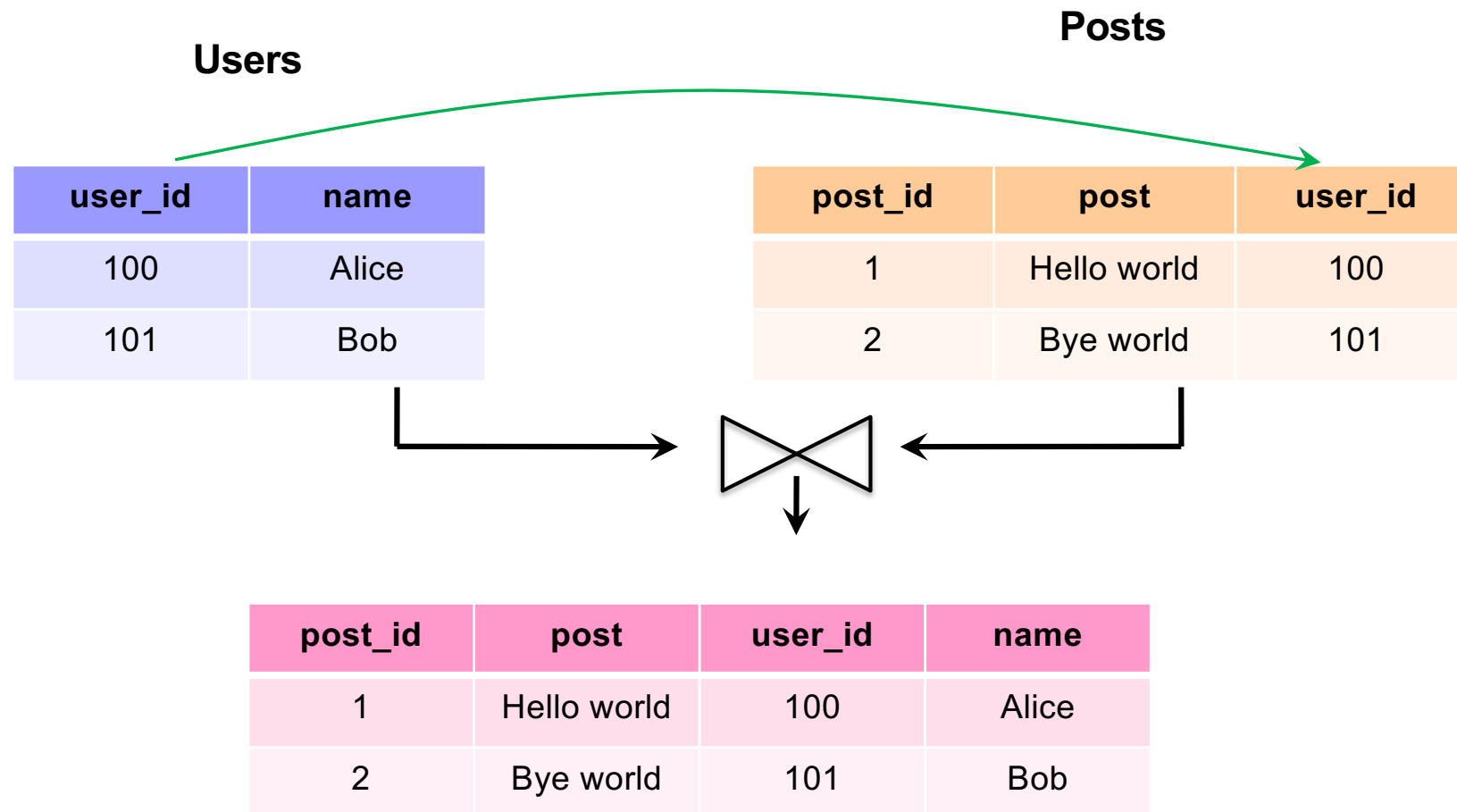
1. True
2. False

Q3: True or false: in MongoDB, shards are beneficial to allow for redundancy.

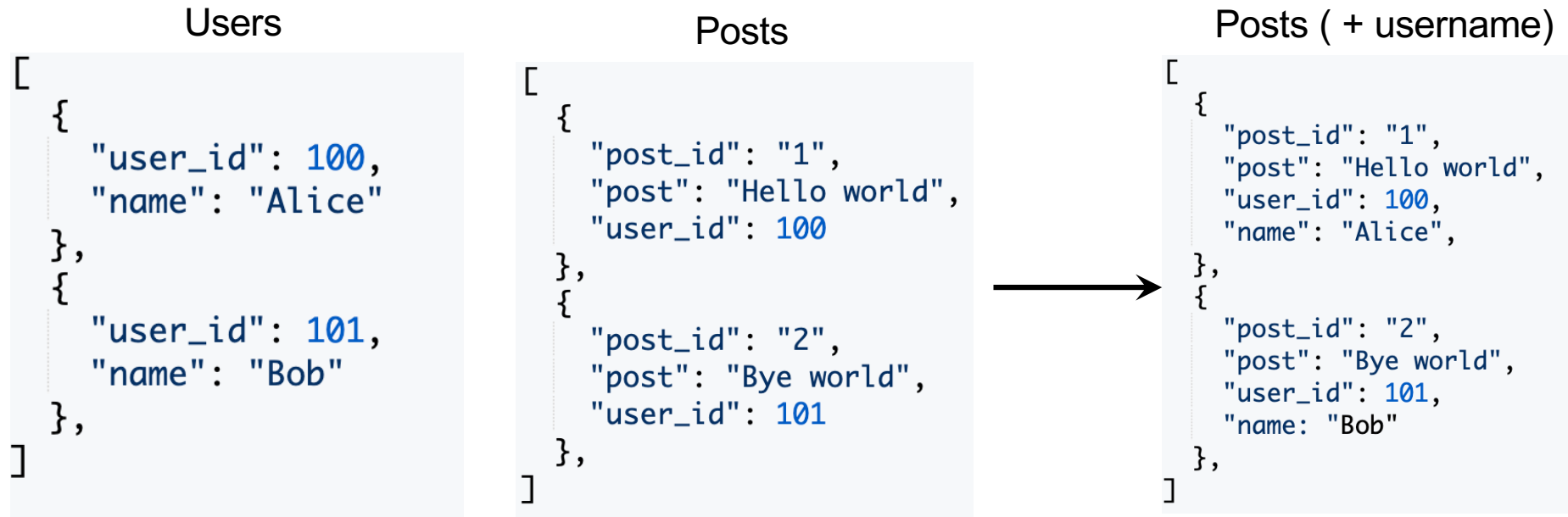
1. True

2. False

Recall: Joins in RDBMS



What if we want to display posts with usernames?



- Answer 1: some NoSQL databases do support joins (e.g. later versions of MongoDB)
- Answer 2: Duplication (i.e. ‘denormalization’)
 - ‘Storage is cheap: why not just duplicate data to improve efficiency?’
 - Tables are designed around the queries we expect to receive. This is beneficial especially when we mostly need to process a fixed type of queries
 - Leads to a new problem: what if user changes their name? (this needs to be propagated to multiple tables)

Conclusion: Pros & Cons of NoSQL Systems

Pros



- + **Flexible / dynamic schema:** suitable for less well-structured data
- + **Horizontal scalability:** we will discuss this more next week
- + **High performance and availability:** due to their relaxed consistency model and fast reads / writes



Cons

- **No declarative query language:** query logic (e.g. joins) may have to be handled on the application side, which can add additional programming
- **Weaker consistency guarantees:** application may receive stale data that needs to be handled by the application